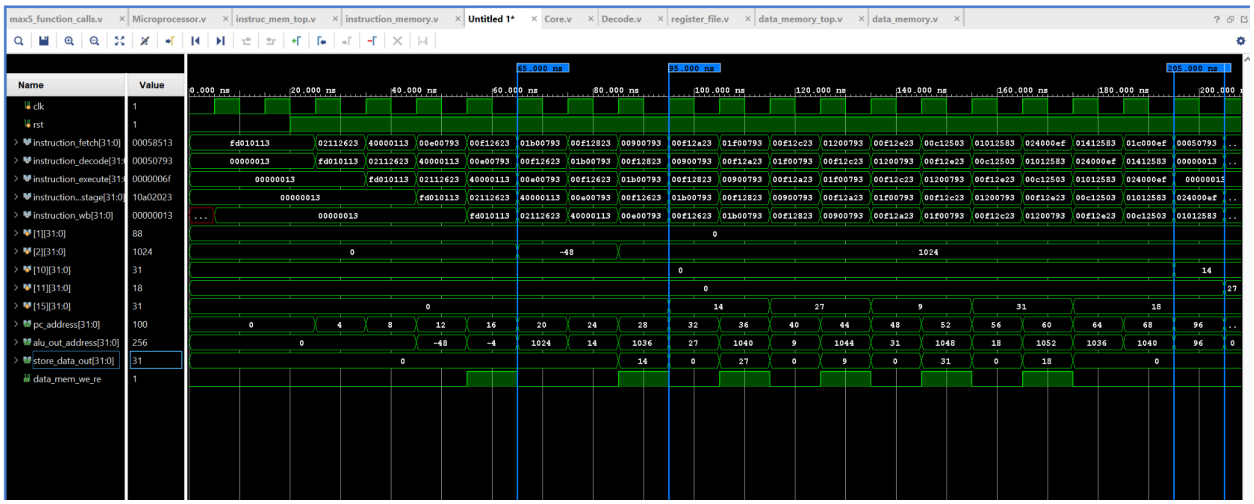
Simulation Results of codes tested:

CODE 1:

Max of 5 numbers using function calls:

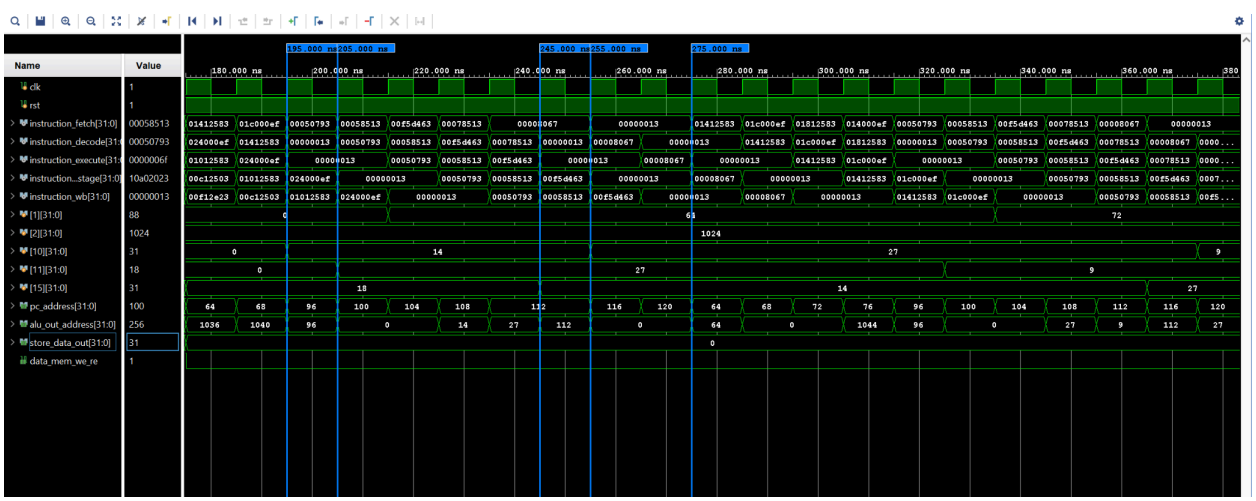
Total clock cycles: 64

(starting from 15ns-655ns)



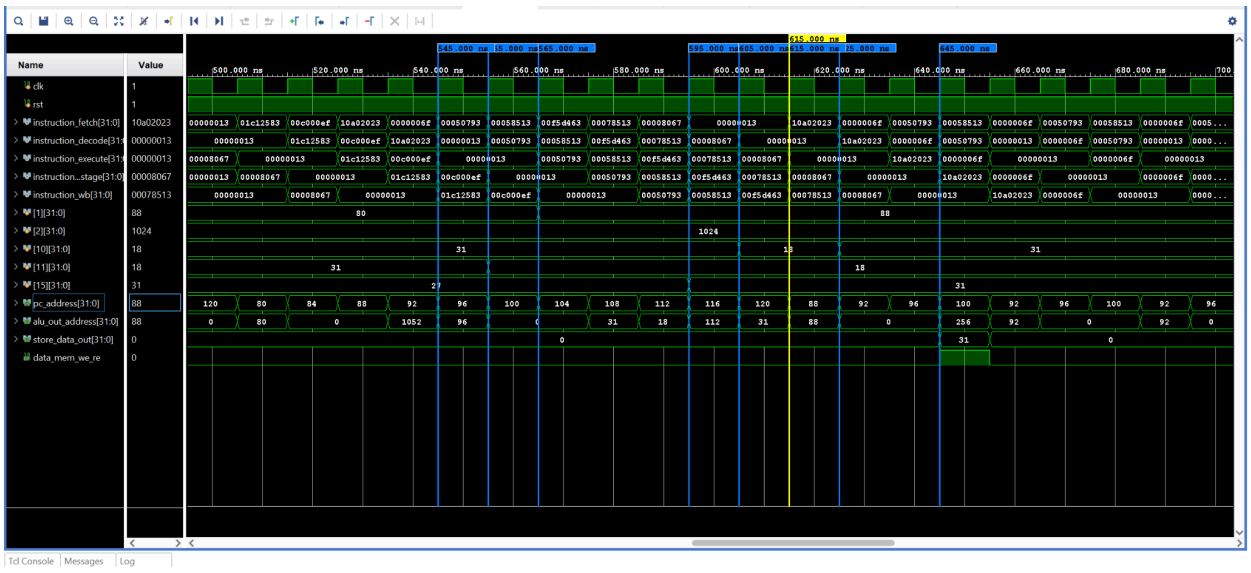
At 65ns (blue marker), sp(x2) is getting the value -48.

From 95ns(blue marker), the 5 values are getting stored in memory through a temporary register a5(x15).



At 195ns (blue marker) and 205ns, we are loading the first two values to a0(x10)=14 and a1(x11)=27 (write back stage)
At 195ns, it is calling the max function so the PC jumps to that address(execute stage).
At 245ns and 255ns, we are moving a0(x10) to a5(x15) and a1(x11) to a0(x10) and seeing if a1>a5. Since 27>14, it branches to skip which has ret and ret has 64, so PC gets updated to that at 275ns. The next instruction(00078513) gets flushed.

After this again when branch happens, the mov instruction moves 27 to a5 so it gets compared with the next value.



At 545ns (blue marker) and 555ns, we are loading the two values to a0(x10)=31 and a1(x11)=18 (write back stage)
At 545ns, it is calling the max function so the PC jumps to that address(execute stage).
At 595ns and 605ns, we are moving a0(x10) to a5(x15) and a1(x11) to a0(x10), and seeing if a1>a5. Since 18<31, it does not branch .
The next instruction(00078513) moves a5 which has 31 to a0 at 625ns.

At 615ns, ret has 88, so the PC gets updated to that.

Finally, the value in a0 gets stored in data memory at 645ns. The store_data_out has value 31 which shows that the maximum value got stored in memory.

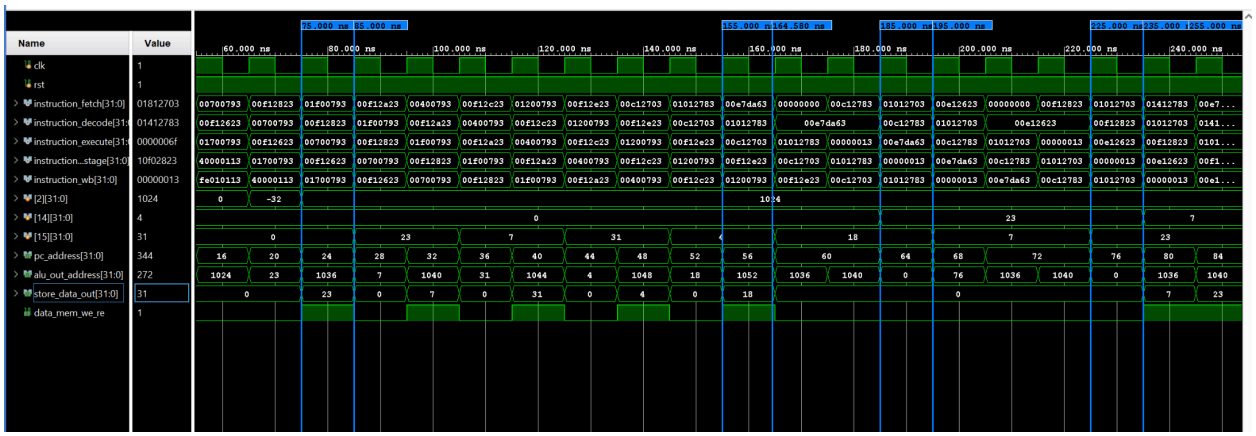
CODE 2:

Sort a list of 5 numbers:

Total clock cycles: 118

(starting from 15ns-1195ns)

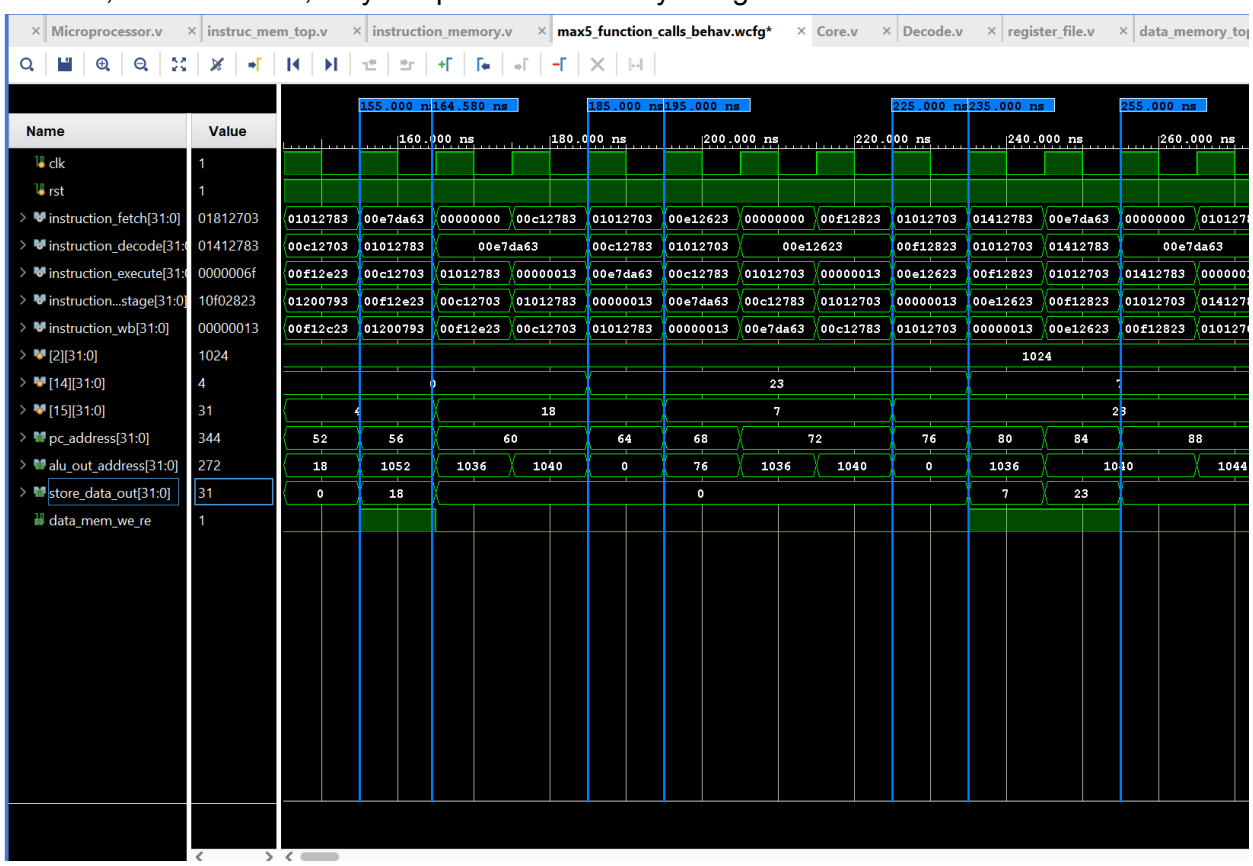
Loading and storing 5 values in data memory by data forwarding (75ns to 165ns)



First comparison of first pass:

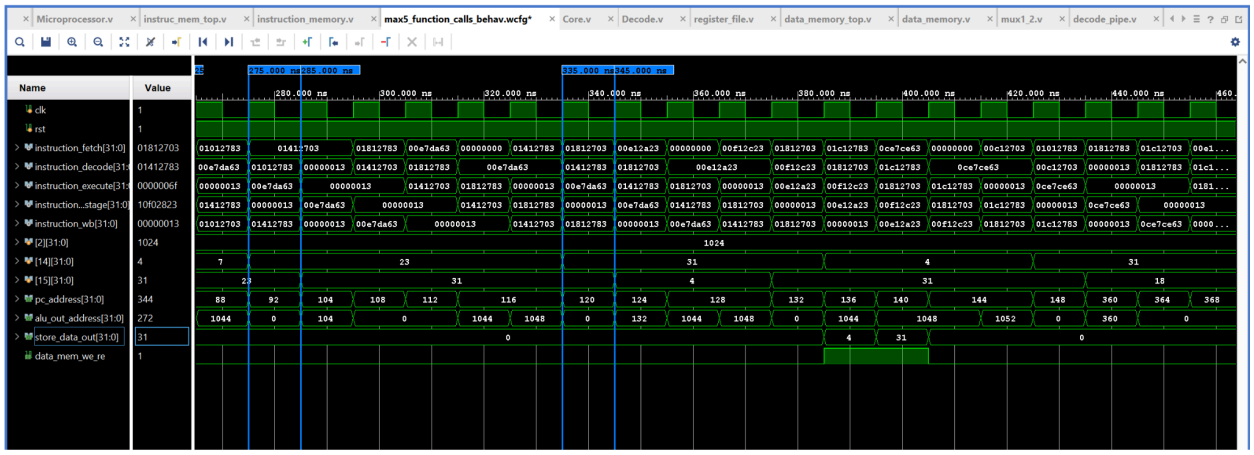
At 185ns and 195ns, 23 and 7 are loaded in a4(x14) and a5(x15).

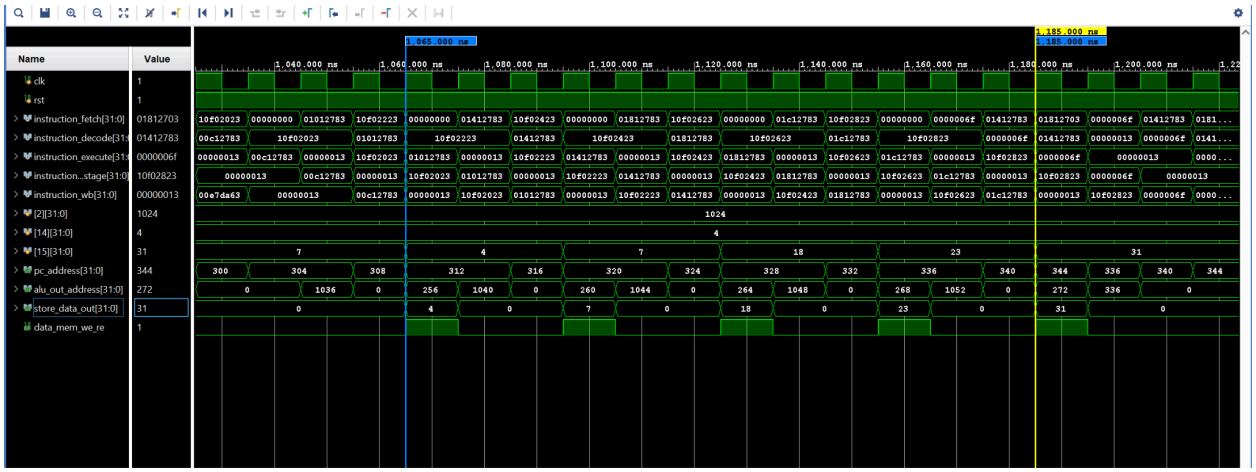
As 23>7, it wont branch, they swap at 225-255ns by using two load and two store instructions.



Second comparison of first pass:
At 275ns and 285ns, 23 and 31 are loaded in a4(x14) and a5(x15).
23<31, it did not swap and branches to the third comparison at 285ns.

In the third pass, we are loading 31 and 4 at 335ns and 345ns.



Similarly it will complete all the passes, we get a sorted list.

Writing back the sorted list to data memory (as we can see from 1065ns to 1185ns):
The final sorted list can be seen from store_data_out as (4,7,18,23,31).